

- È un demone (internet daemon) associato a più porte che attende connessioni in ingresso. Quando arriva una connessione, genera un nuovo processo ed esegue il demone appropriato.

79. Quali caratteristiche hanno le connessioni TCP (full-duplex, punto a punto)?

- Full-duplex: traffico in entrambe le direzioni contemporaneamente. Punto a punto: ogni connessione ha esattamente due punti terminali (non supporta multicast o broadcast).

80. A cosa serve il flag PUSH in TCP?

- Permette alle applicazioni di forzare l'uscita immediata dei dati senza buffering. Oggi si usa TCP_NODELAY.

81. A cosa servono i dati urgenti (flag URG) in TCP?

- Per dati ad alta priorità (es. CTRL-C per interrompere un processo remoto). Il flag URG e l'Urgent pointer indicano dove si trovano i dati urgenti.

82. Quanti bit ha il numero di sequenza di un byte in TCP?

- 32 bit.

83. Quali sono i due limiti sulla dimensione di un segmento TCP?

- Deve essere contenuto nel payload IP (65.515 byte max) e nella MTU di ogni collegamento (per evitare frammentazione).

84. Qual è il protocollo di base utilizzato da TCP?

- Protocollo a finestra scorrevole con dimensione dinamica della finestra.

Sottosezione 7.4.2 – Intestazione del Segmento TCP

85. Quali sono i campi principali dell'intestazione TCP?

- Source port, Destination port, Sequence number, Acknowledgement number, TCP header length, Flags (CWR, ECE, URG, ACK, PSH, RST, SYN, FIN), Window size, Checksum, Urgent pointer, Options.

86. Perché l'identificatore di connessione TCP è chiamato "quintupla"? Quali cinque elementi lo compongono?

- Protocollo (TCP), IP sorgente, porta sorgente, IP destinazione, porta destinazione.

87. Il campo Acknowledgement number in TCP è cumulativo? Cosa significa?

- Sì, specifica il successivo byte previsto (non l'ultimo ricevuto), riassumendo tutti i dati ricevuti con un solo numero.

88. A cosa serve il campo TCP header length?

- Indica quanti gruppi di 32 bit sono contenuti nell'intestazione TCP (perché Options ha lunghezza variabile).

89. Quali sono i flag CWR e ECE e a cosa servono?

- Usati per segnalare congestione con ECN (Explicit Congestion Notification). ECE: ECN-Echo al mittente per rallentare. CWR: Congestion Window Reduced dal mittente.

90. A cosa serve il flag URG e il campo Urgent pointer?

- URG indica che ci sono dati urgenti; Urgent pointer indica lo spiazamento in byte dove si trovano.

91. Cosa indica il flag ACK?

- Indica che il campo Acknowledgement number è valido. Quasi tutti i pacchetti lo hanno settato.

92. A cosa serve il flag PSH?

- Richiede al ricevente di consegnare i dati all'applicazione all'arrivo, senza bufferizzare.

93. Cosa fa il flag RST?

- Reimposta una connessione confusa (malfunzionamento) o rifiuta un segmento non valido o un tentativo di connessione.

94. Qual è la differenza tra un segmento SYN con ACK=0 e uno con ACK=1?

- SYN+ACK=0: richiesta di connessione (CONNECTION REQUEST). SYN+ACK=1: risposta alla richiesta (CONNECTION ACCEPTED).

95. A cosa serve il flag FIN?

- Rilascia una connessione: indica che il mittente non ha altri dati da trasmettere.

96. Come viene gestito il controllo di flusso in TCP?

- Con finestra scorrevole a dimensione variabile (campo Window size). Indica quanti byte possono essere inviati a partire da quello che ha ricevuto acknowledgement.

97. Cosa significa un campo Window size pari a 0?

- Che il ricevente ha ricevuto i byte fino ad Acknowledgement number-1, ma non ha spazio per altri dati per ora.

98. Il checksum in TCP è obbligatorio? Cosa verifica?

- Sì, obbligatorio. Verifica intestazione, dati e pseudointestazione (con numero di protocollo TCP=6).

99. A cosa serve l'opzione MSS (Maximum Segment Size)?

- Permette a ogni host di specificare la dimensione massima del segmento che è disposto ad accettare.

Sottosezione 7.4.3 – Instaurazione di una Connessione

100. Come viene stabilita una connessione TCP? Descrivi il three-way handshake.

- Client invia SYN (seq=x). Server risponde con SYN+ACK (seq=y, ack=x+1). Client invia ACK (ack=y+1). Connessione stabilita.

101. Cosa succede se un segmento SYN arriva su una porta dove nessun processo è in ascolto?

- Viene inviata una risposta con il bit RST=1 per rifiutare la connessione.

102. Descrivi il caso di instaurazione simultanea di una connessione da entrambi i lati.

- Entrambi inviano SYN con ACK=0. Entrambi rispondono con SYN+ACK. Viene creata una sola connessione (identificata dalla stessa coppia di socket).

103. Perché un segmento SYN consuma un byte nello spazio dei numeri di sequenza?

- Per poter ricevere un acknowledgement non ambiguo che confermi la ricezione del SYN stesso.

Sottosezione 7.4.4 – Rilascio di una Connessione**104. Perché è utile immaginare una connessione TCP come due connessioni simplex separate?**

- Perché ogni direzione può essere rilasciata indipendentemente: una parte può chiudere la sua direzione ma continuare a ricevere dati.

105. Quanti segmenti sono normalmente necessari per rilasciare una connessione TCP? Perché a volte bastano tre?

- Normalmente 4 (FIN, ACK, FIN, ACK). A volte 3 se il primo ACK e il secondo FIN sono nello stesso segmento.

106. Come si evita il problema dei due esercizi nel rilascio di una connessione TCP?

- Usando timer: se una risposta a FIN non arriva entro il tempo di vita massimo di due pacchetti, il mittente del FIN rilascia comunque la connessione.

Sottosezione 7.4.5 – Modello di Gestione della Connessione**107. Quali sono i principali stati della macchina a stati finiti TCP? Descrivili brevemente.**

- CLOSED (nessuna connessione), LISTEN (in attesa), SYN RCVD (richiesta ricevuta), SYN SENT (richiesta inviata), ESTABLISHED (normale), FIN WAIT 1/2 (chiusura iniziata), TIME WAIT (attesa pacchetti), CLOSE WAIT (altro lato ha iniziato chiusura), LAST ACK, CLOSING.

108. Qual è il percorso normale degli stati per un client? E per un server?

- Client: CLOSED → SYN SENT → ESTABLISHED → FIN WAIT 1/2 → TIME WAIT → CLOSED. Server: CLOSED → LISTEN → SYN RCVD → ESTABLISHED → CLOSE WAIT → LAST ACK → CLOSED.

109. In quale stato un server attende connessioni in ingresso?

- LISTEN.

110. In quale stato avviene il normale trasferimento dei dati?

- ESTABLISHED.

Sottosezione 7.4.6 – La Finestra Scorrevole**111. Cosa sono i delayed acknowledgement (acknowledgement ritardato)? Perché sono utili?**

- Ritardare gli acknowledgement e gli aggiornamenti della finestra (tipicamente 500 ms) per aggregarli con dati in uscita. Riduce il numero di pacchetti e l'utilizzo della banda.

112. In cosa consiste l'algoritmo di Nagle? Quale problema risolve?

- Inviare il primo gruppo di dati piccoli, bufferizzare i successivi fino all'arrivo dell'acknowledgement. Riduce lo spreco di banda causato da molti pacchetti piccoli.

113. Cos'è la silly window syndrome (sindrome della finestra stupida)? Come si manifesta?

- Quando il ricevente legge 1 byte alla volta, inviando aggiornamenti della finestra per 1 byte, e il mittente invia segmenti da 1 byte. Causa inefficienza.

114. Qual è la soluzione di Clark alla silly window syndrome?

- Il ricevente invia aggiornamenti della finestra solo quando può gestire la dimensione massima del segmento o quando il buffer è vuoto per metà.

Sottosezione 7.4.7 – Controllo della Congestione**115. Come interpreta TCP la perdita di pacchetti?**

- Come segnale binario di congestione.

116. Cos'è la finestra di congestione (congestion window)? Come si relaziona con la finestra di controllo del flusso?

- È il numero di byte che il mittente può avere sulla rete. La finestra effettiva è la minima tra finestra di congestione e finestra di controllo del flusso.

117. In cosa consiste l'algoritmo di slow start? Perché si chiama così se ha crescita esponenziale?

- La finestra di congestione inizia a 1 segmento e raddoppia a ogni RTT. Si chiama "lenta" rispetto all'algoritmo precedente che inviava un'intera finestra subito.

118. Cos'è la slow start threshold? Quando viene aggiornata?

- Soglia che limita lo slow start. Quando si verifica una perdita (timeout), la soglia viene impostata a metà della finestra di congestione corrente.

119. Cosa succede quando viene rilevata una perdita di pacchetto (timeout)?

- La slow start threshold viene impostata a metà della finestra corrente, la finestra di congestione viene reimpostata a 1 segmento, e ricomincia lo slow start.

120. In cosa consiste la fase di incremento additivo (additive increase)?

- Dopo aver superato la soglia, la finestra di congestione viene aumentata di 1 segmento per ogni RTT (crescita lineare).

121. Cosa sono TCP Tahoe e TCP Reno?

- Versioni di TCP con controllo della congestione. Tahoe: slow start + incremento additivo. Reno: aggiunge fast recovery e fast retransmit.

122. Cos'è il fast recovery? Quando viene utilizzato?

- Modalità temporanea che mantiene l'ack clock in funzione dopo una fast retransmit, evitando lo slow start. La finestra viene ridotta a metà.

123. Quale profilo segue la finestra di congestione di TCP con AIMD?

- Profilo a "dente di sega": incrementi additivi (di 1 segmento per RTT) e decrementi moltiplicativi (di metà in un RTT dopo una perdita).

124. Cosa sono i SACK (Selective Acknowledgement)? Quale vantaggio offrono?

- Elencano fino a tre intervalli di byte ricevuti. Permettono al mittente di ritrasmettere solo i pacchetti realmente persi, non l'intera finestra.

Sottosezione 7.4.8 – Gestione dei Timer**125. Qual è il timer più importante in TCP? A cosa serve?**

- Il timer di ritrasmissione (RTO – Retransmission Timeout). Quando scade prima dell'ACK, il segmento viene ritrasmesso.

126. Cos'è il RTO (Retransmission Timeout)?

- L'intervallo di timeout dopo il quale un segmento non riconosciuto viene ritrasmesso.

127. Come viene calcolato dinamicamente il RTO? Cosa sono SRTT e la formula EWMA?

- $SRTT = \alpha \cdot SRTT + (1-\alpha) \cdot R$ (con $\alpha=7/8$). È una media mobile a peso esponenziale (EWMA) che stima il round-trip time corrente.

128. Perché il timeout in TCP deve essere adattivo e non fisso?

- Perché le prestazioni di rete variano nel tempo (congestione, percorsi diversi, carico). Un timeout fisso sarebbe troppo breve (ritrasmissioni inutili) o troppo lungo (scarsa reattività).

Sottosezione 7.4.9 – WebRTC**129. Cosa significa l'acronimo WebRTC?**

- Web Real-Time Communication.

130. Quale protocollo di trasporto utilizza WebRTC per la comunicazione dati? Perché?

- TCP (in aggiunta a UDP per media). TCP garantisce affidabilità e ordine corretto, essenziale per applicazioni real-time.

131. Quali sono i principali pregi di WebRTC?

- Bassa latenza, architettura P2P (riduce latenza e migliora qualità), compatibilità con browser moderni.

132. Quali sono i principali difetti di WebRTC?

- Complessità di configurazione, limitazioni di sicurezza (vulnerabilità in implementazioni non corrette), requisiti di rete (connessione stabile e ad alta velocità).

Sezione 7.5 e 7.6 – LAB (Socket e Sistemi Client-Server)

133. A cosa serve una socket nella programmazione di rete?

- È un punto terminale di comunicazione che permette a un processo di inviare e ricevere dati attraverso la rete.

134. Quali sono le fasi tipiche di implementazione di un sistema client-server con socket?

- Server: socket(), bind(), listen(), accept(), recv()/send(), close(). Client: socket(), connect(), send()/recv(), close().

Domande trasversali e di confronto

135. Confronta UDP e TCP in termini di: orientamento alla connessione, affidabilità, controllo di flusso, controllo di congestione, dimensione intestazione, campi principali.

Caratteristica	UDP	TCP
Orientamento connessione	No (connectionless)	Sì (connection-oriented)
Affidabilità	Nessuna (best effort)	Affidabile (ACK, ritrasmissioni)
Controllo di flusso	No	Sì (finestra scorrevole)
Controllo congestione	No	Sì (AIMD, slow start)
Dimensione intestazione	8 byte	20-60 byte
Campi principali	Source port, Dest port, Length, Checksum	Source/dest port, Seq/ACK num, Flags, Window, Checksum

136. Quali sono le differenze tra il controllo degli errori nel livello data link e nel livello trasporto?

- Data link: protegge un singolo collegamento (hop-by-hop). Trasporto: protegge l'intero percorso (end-to-end). Il checksum del trasporto è essenziale per la correttezza; quello del data link è utile per le prestazioni.

137. Confronta slow start e incremento additivo in termini di crescita della finestra di congestione.

- Slow start: crescita esponenziale (raddoppia ogni RTT). Incremento additivo: crescita lineare (aumenta di 1 segmento per RTT).

138. Spiega la differenza tra il three-way handshake per stabilire una connessione e il three-way handshake per rilasciarla.

- Setup: SYN, SYN+ACK, ACK (stabilisce numeri di sequenza). Rilascio: FIN, ACK+FIN, ACK (chiusura di ciascuna direzione). Il rilascio può richiedere 4 segmenti (o 3 se combinati).

139. Qual è la differenza tra multiplexing e inverse multiplexing nel livello di trasporto?

- Multiplexing: più connessioni di trasporto condividono lo stesso NSAP (stesso IP). Inverse multiplexing: una connessione di trasporto distribuisce il traffico su più percorsi di rete.

140. Confronta RTP e RTCP in termini di funzionalità e scopo.

- RTP: trasporta dati multimediali (audio/video) con timestamp e sequenze, senza ritrasmissioni. RTCP: trasporta informazioni di controllo (feedback su congestione, jitter, sincronizzazione tra flussi, identificazione delle sorgenti).